

Operating Systems: Test 1

Computer Systems Overview · Memory Management · Process Management. 50 min, one letter sheet both sides.

Every formula	
Quantity	Formula
Hit ratio	$H = \text{hits} / \text{total accesses}$; $\text{miss} = 1 - H$
Avg access time (2-level)	$T_s = H \cdot T_1 + (1 - H) \cdot (T_1 + T_2) = T_1 + (1 - H) \cdot T_2$
Access efficiency	$T_1 / T_s = 1 / (1 + (1 - H) \cdot r)$, $r = T_2 / T_1$
Avg cost per bit	$C_s = (C_1 \cdot S_1 + C_2 \cdot S_2) / (S_1 + S_2)$ want $C_s \leq C_2 \implies \text{need } S_1 \leq S_2$
TLB effective access time	$EAT = h(t_t + t_m) + (1 - h)(t_t + 2t_m)$
Page table size	offset bits = $\log_2(\text{page size})$ page bits = $\text{addr bits} - \text{offset bits}$ size = $2^{\text{page bits}} \times \text{entry size}$
Address space	$2^{\text{addr bits}}$; #pages = $2^{\text{addr} / 2^{\text{offset}}}$
CPU utilization	$\text{execute} / (\text{read} + \text{execute} + \text{write})$
First-fit searches	$(N+1)/2$ for N equally-likely blocks

Worked numbers

Page table. 32-bit addr, byte addressing, 4 KB=2¹² pages, 4 B entries: 2³²/2¹² = 2²⁰ pages; 2²⁰×4 B = **4 MB**/process. 100 processes = 400 MB.
AMAT. T₁=0.1 μs, T₂=1 μs, H=0.95: 0.95(0.1)+0.05(1.1) = **0.15 μs**, near the fast level.
TLB. t_t=20, t_m=100 ns, h=0.8: 0.8(120)+0.2(220) = **140 ns**. (Miss = page table *then* data.)
CPU util. read 15 μs + exec 1 + write 15 = 31; 1/31 = **3.2%** (why multiprogramming exists).
Paging bits. 15-bit logical addr, 5 page bits: space 2¹⁵=32 KB, 2⁵=32 pages, page = 2¹⁰=1 KB.

I · COMPUTER SYSTEMS OVERVIEW

Basic elements

- **Processor** (CPU) controls operation + processes data. **Main memory** stores data+programs, **volatile** (= real/primary memory). **I/O modules** move data in/out. **System bus** connects all three.
- In RAM one row = **frame**; its length = **word**.
- Registers: **PC** (addr of next instr), **IR** (fetched instr), **MAR** / **MBR** (memory addr/buffer), I/O AR + I/O BR.
- Variants: **microprocessor** (CPU on one chip, now multi-core), **GPU** (SIMD arrays), **DSP** (streaming signals, codecs), **SoC** (CPU+cache+GPU+DSP+controllers on one chip).

Instruction cycle

fetch → **execute** → (**interrupt check**). Fetch instr at PC, PC increments, instr lands in IR, processor interprets and acts. **opcode** = binary code identifying an instruction.
Four action categories: processor ↔ memory, processor ↔ I/O, **data processing** (arith/logic), **control** (branch).

Interrupts

- **Why:** I/O is far slower than the CPU; without interrupts the CPU idles waiting. Interrupts improve processor utilization.
- **How:** user program suspended → **interrupt handler** (part of the OS) runs → process resumes *at the same point*. The user program contains **no special code**: processor + OS do it all.
- An **interrupt stage** is added to the cycle; skipped if interrupts are disabled. The check comes *after* execute because the current instruction must finish.

Type	Source
Program	from executing an instruction: overflow, divide by zero, illegal instruction, reference outside allowed memory
Timer	processor timer: lets the OS act at regular intervals
I/O	I/O controller: completion or error
Hardware failure	power failure, memory parity error

PSW & modes

PSW (program status word) = status of the running process: condition codes, memory info, **interrupt enable/disable bit**, **kernel/user-mode bit**.

User mode	Kernel mode
user programs run here; some memory protected, some instructions forbidden	monitor/kernel runs here; privileged instructions allowed, protected memory accessible

Evolution of OSs

serial processing (no OS, direct hardware, awful scheduling + setup) → **simple batch** (resident **monitor** sequences jobs; job returns control to monitor) → **multiprogrammed batch** (run another job during I/O waits) → **time sharing** (many terminals; OS interleaves short bursts = **quanta**).
Monitor needs 4 hardware features: memory protection (user can't touch the monitor), timer (no job monopolizes), privileged instructions (monitor only), interrupts (flexibility).

	Batch multiprog.	Time sharing
Objective	maximize processor use	maximize response time
Directives	job control language with the job	commands typed at terminals

OS managers: memory, processor, device, file, network.
Generations: 1940–50 no OS · 1950–60 mainframes, batch, multiprogramming · 1960–70 multiprogramming + timesharing · 1970–80 personal computing, GUI · 1980–2000 **microkernels** · 2000– mobile, cloud/virtualization, distributed, real-time/embedded.
Uniprogramming cost: read 15 μs, execute 100 instrs = 1 μs, write 15 μs → 31 μs total, CPU busy 1/31 = **3.2%**. Multiprogramming runs another process during the waits.

Memory hierarchy

Trade-off: faster = higher cost/bit; bigger = lower cost/bit but slower. Layers: **inboard** (registers, cache, main memory) → **out-board** (disk, optical) → **offline** (tape).
Going down (A–D): A cost/bit ↓, B capacity ↑, C access time ↑, D frequency of processor access ↓. **D is what makes it work**, with D, average access time sits near the *fast* level while average cost sits near the *cheap* level.
Principle of locality: references **cluster**: execution is mostly sequential, instruction references stay localized over short periods, and arrays/records are processed element by element. **Temporal** = same location again soon; **spatial** = nearby locations soon. Locality is why caching *and* paging work.

Secondary memory: non-volatile, external, cheaper, stores programs + data.

Cache

- Sits between CPU and main memory; **largely invisible to the OS** (all in hardware). CPU accesses memory ≥ once per instruction cycle, so execution is limited by memory cycle time, cache hides that.
- **Design dimensions:** cache size, block size, mapping function, replacement algorithm, write policy, number of levels.
- **Block** = unit exchanged with main memory. Bigger blocks *first raise* the hit ratio (spatial locality), then *lower* it (much of the fetch is unused, and big blocks evict useful data) → there is a sweet spot.

Mapping	Where a block may go
Direct	exactly one slot, e.g. bBlock mod 10. Simplest, most conflicts.
Fully associative	any slot. Flexible, fewest conflicts, but must search/compare many entries; costly circuitry.
Set associative	one set, any slot in it. The compromise (e.g. 10 slots as 5 sets of 2).

Replacement: LRU: replace the block resident longest with no references. **Write policy:** write on every update, or write back on replacement (fewer writes, but main memory temporarily obsolete). Write-through helps when **multiple processors share memory**: all see consistent data without coherence protocols.

II · MEMORY MANAGEMENT

Uni- vs multiprogramming

Uni: one program, whole thing loaded contiguously, memory = OS + one user program. Wasteful: size capped by memory, unused allocation wasted, CPU idle on I/O. **Multi:** divide user memory among processes so the CPU switches to another on a wait. **Memory management** decides how memory is allocated to and used by processes.

Five requirements

Relocation	Load a program into any free part of memory and still address it correctly. Needed because the programmer can't know the load address, and a swapped-out process may return to a <i>different</i> region. Translation happens at run time.
Protection	Other processes can't touch this process's memory without permission. Location is unpredictable ⇒ check references at run time , and it must be hardware , the OS can't anticipate every reference. Schemes supporting relocation also support protection.
Sharing	Several processes access the same memory in a controlled way (one copy of a shared library) without breaking protection.
Logical org.	Programs are modules (some modifiable, some not); hardware is a flat 1-D address space. Modules allow independent compilation, per-module protection, module sharing. Segmentation supports this best.
Physical org.	Manage the two-level memory and the flow between levels. Must be the system's job, not the programmer's: main memory may be too small, and the programmer can't know available space (overlays were the old manual answer).

Types of address

Physical (absolute)	actual location in main memory
Logical	a reference <i>independent</i> of the current assignment of data to memory
Relative	a logical address expressed relative to a known point (the program origin)

Hardware translates relative → physical with **base register** (start address) and **bounds register** (end address: also enforces protection).

Fixed partitioning

Static partitions set at system-generation time; a process loads

into a partition ≥ its size.

- **Equal-size:** placement trivial (any free partition). **Unequal-size:** assign to the **smallest partition that fits**; one queue per partition, or a single queue (better utilization, needs relocation).
- Program bigger than a partition → **overlays**.
- **Problems: internal fragmentation** (unused space inside the partition is wasted), inefficient memory use, and a hard cap on the number of active processes.
- **Deallocation:** just reset the partition's free/busy bit.

Dynamic partitioning

Variable number and length of partitions; each process gets *exactly* what it needs. OS tracks partition boundaries.

- Kills internal fragmentation, creates **external fragmentation**: many small unusable holes between processes.
- **Compaction** shifts processes together into one contiguous free block. Expensive (copies all resident processes, halts work) and **requires run-time relocation**.
- **Deallocation:** must **coalesce** adjacent free areas.

Placement	Rule / note
Best-fit	block closest in size to the request. O(n).
First-fit	first hole from the start that fits. Worst: leaves small holes near the front, forcing compaction sooner.
Next-fit	next hole that fits, scanning <i>from the last placement</i> , wrapping around. Best of the three .

Buddy system: combination of fixed + dynamic. Blocks are always 2^K, L ≤ K ≤ U. Request s takes 2^K where 2^{K-1} < s ≤ 2^K; if none free, **split** the smallest larger block in half repeatedly (the halves are **buddies**). On release, **merge** with the buddy if it is free, repeatedly upward. *1 MB, request 100 KB:* 1M→512K→256K→128K, allocate 128 KB (28 KB internal frag). Less external frag than dynamic, less internal than fixed, but rounds every request up to a power of 2.

Relocation: fixed vs dynamic

- **Dynamic** → **always** required (swap-in lands in a different hole; compaction shifts processes).
- **Fixed** → **only** if a process may be reassigned to a *different* partition (single queue over unequal partitions). Always returns to its own partition ⇒ load address never changes ⇒ no relocation.
- Either way the referenced addresses are **not fixed** ⇒ resolve at run time via base/bounds.

Paging

- Memory in equal fixed-size **frames**; process in equal **pages** of the same size. Pages go into any free frames, **need not be contiguous** ⇒ **no external fragmentation**.
- Waste = internal fragmentation only, at most a fraction of the **last page**.
- **Transparent to the programmer.** With a power-of-2 page size the logical address = the relative address, and the physical address is formed by **appending the offset to the frame number** (no arithmetic).
- OS keeps one **page table** per process: **page number** → **frame number**. Terms: free chunks = **holes**/frames; chunks of a process = **pages**.
- **Cost:** 2 memory accesses per reference (page table, then data), so paging **doubles** access time. A page fault adds disk I/O.

Segmentation

Process split into **segments** of *varying* length (up to a max), matching logical modules. Address = (segment #, offset); **segment table** holds each segment's **base + length**.

- **Visible** to the programmer. **Eliminates internal** fragmentation; suffers **external** fragmentation. Segments need *not* be the same length.
- Advantages: handles growing data structures; modules recompiled independently; natural sharing and protection per module.

Paging + segmentation

Split the space into segments, then each segment into pages. Translation is two-level: segment table → that segment's page table → frame. Gets paging's efficiency plus segmentation's modularity/sharing/protection, at the cost of two tables and two lookups.

Why paging won: (1) simpler memory management; (2) better fit for virtual memory, since variable-size segments make swapping and protection trickier; (3) more granular, more consistently enforceable protection; (4) better match to modern hardware.

Virtual memory

Virtual memory	uses secondary storage as if it were RAM; size capped by the addressing scheme + available disk , not by RAM
Virtual / real address	address in virtual memory / actual location in main memory
Virtual address space	the virtual memory assigned to one process
Resident set	the portion of a process that <i>is</i> in main memory

Two breakthroughs make it possible: (1) all references are logical addresses translated dynamically at run time; (2) a process can be broken into pieces that need not be contiguous. Given both, *not all* pieces need be resident.

On a reference to a non-resident page: **page fault** interrupt → OS puts the process in **Blocked** → issues a disk read → **dispatches another process** → I/O-completion interrupt → process back to **Ready**.
Implications: more processes resident (only parts loaded) ⇒

more Ready processes $\implies$ better CPU use; and a process may be **larger than all of main memory** (no programmer overlays).
Requires: hardware support for paging/segmentation, and OS software to move pieces between the levels.
Thrashing: the system spends most of its time swapping pieces rather than executing. Caused by evicting a piece just before it is needed; the OS **guesses from recent history** which pieces are least likely to be used soon. Works because of **locality**: over a short period a process needs only a small subset of its pages. Load control (degree of multiprogramming) is the knob: too many resident processes $\implies$ too few frames each $\implies$ thrash.

Page tables at scale

A flat table grows with the *address space* and is paid *per process*, no matter how little memory is used (4 MB each; 400 MB for 100 processes). Three fixes:

- **Two-level / hierarchical:** page the page table; only the **root table is always resident**, the rest lives in virtual memory. Modern chips do 4–5 levels.
- **Inverted page table:** one entry per **real frame** $\implies$ size bounded by **physical** memory, one table system-wide. Indexed by frame, so a virtual page number can't index it; a **hash of (process id, page #)** locates the entry. Entry = page #, process id, control bits, **chain pointer** (collisions). "Inverted" = indexed by frame, not by page.
- **TLB** (translation lookaside buffer): high-speed cache of recent page-table entries (page # + the full entry). **TLB hit** $\rightarrow$ frame number directly, no page-table access. All entries are compared **at once** (**associative mapping**); a scan would cost more than it saves, so the TLB is small. Older CPUs flush it on a context switch; modern ones tag entries (ASID) and keep several processes' pages.

Page size

- **Smaller:** less internal fragmentation, but a *bigger* page table, which may itself be paged $\implies$ possible **double page fault** (page-table page, then the data page).
- **Larger:** better transfers from rotational secondary storage.
- Also: main memory size and program size; OO code can *reduce* locality (many small scattered routines).
- **Fault rate vs page size:** tiny pages keep much of the locality resident (few faults) $\rightarrow$ growing pages hold locations further from recent references, diluting locality (faults **rise** to a max) $\rightarrow$ very large pages approach the whole process, and when one page spans it, faults $\rightarrow$ 0. (P = process size, W = working set, N = total pages.)

OS policies

Fetch	Demand paging: bring a page in only when referenced (many faults at startup, then locality drops the rate). Prepaging: bring in neighbours too, since contiguous disk reads are cheaper per page; wasted if unreferenced. <i>Not</i> swapping.
Placement	Matters for segmentation (which hole?); irrelevant for paging: hardware treats every frame equally.
Replacement	Pick the page least likely to be referenced soon . More elaborate policy = more overhead. Frame locking: a locked frame can't be replaced: kernel, key control structures, I/O buffers, time-critical code; one lock bit per frame.
Resident set size	Fixed allocation (a fault replaces one of the process's own pages) vs variable (count changes over its life, more powerful, but the OS must assess behaviour).
Replacement scope	Local: only the faulting process's resident pages. Global: all unlocked frames in memory (can take another process's page). Both triggered by a fault with no free frames.
Cleaning	Demand vs precleaning (when to write a modified page back).
Load control	The degree of multiprogramming.

Resident set trade-off: fewer frames per process $\implies$ more processes resident but more faults each; past a certain size, extra frames barely change the fault rate.

Replacement algorithms

Reference string 2 3 2 1 5 2 4 5 3 2 5 2, **3 frames:**

Algorithm	Evicts	After fill	Total
Optimal	page referenced furthest in the future	3	6
LRU	page unreferenced the longest	4	7
FIFO	page resident the longest , used or not	6	9
Clock	FIFO but skipping use bit = 1 (clearing it)	5	8

Optimal needs the future $\implies$ not implementable; it is the lower bound others are judged against. **Clock** = cheap LRU: frames in a circular buffer with a pointer; a reference sets the use bit to 1; on a fault the pointer advances, resetting 1s to 0, and replaces the first frame already at 0. **Watch the convention:** faults that fill the empty frames count in the total but are *not* marked F in Fig. 8.14.

III · PROCESS MANAGEMENT

Program vs process

A **program** is passive (instructions in a file). A **process** is a program brought to life: an *instance running* (user view); an *entity schedulable on a processor* (scheduler view); a *unit of activity* = instructions + current state + resources (implementer view).

Elements the OS tracks (8): identifier, state, priority, program counter, memory pointers, context data, I/O status, accounting

info.

PCB and process image

All of the above lives in the **process control block**. On an interrupt: PC + registers (the **context**) are saved to the PCB, the state changes, another process runs. Because nearly every OS module reads/writes PCBs, they effectively **define the OS's state** and must be protected (kernel memory, kernel mode).

PCB category	Contents
Process identification	this PID, parent PID, user id
Processor state info	user-visible registers, control/status registers, PSW , stack pointers
Process control info	state, priority, scheduling info, the event awaited , memory pointers, privileges, resources, linkage to other PCBs

Process image = **user program + user data + stack + PCB**. (Stack is LIFO, for call parameters and return addresses.) Under simple schemes it is a contiguous block that must be **fully** resident; under virtual memory only part need be.

Trace and dispatcher

Trace = the sequence of instructions one process executes. The **processor's** behaviour = those traces **interleaved**. **Dispatcher** = the small program that switches the processor between processes.

A at 5000, B at 8000, C at 12000, dispatcher at 100 (6 instrs), timeout after 6, B does I/O after 4:

Instrs	Running	Stopped by
5000–5005	A	timeout
100–105	dispatcher	
8000–8003	B	I/O $\rightarrow$ Blocked
100–105	dispatcher	
12000–12005	C	timeout
100–105	dispatcher	
5006–5011	A	timeout

Reading a trace: a **timeout** uses the whole slice and leaves the process **Ready** (dispatched again in turn); a **blocking event** stops it **early** and sends it to **Blocked** (not dispatched until the event occurs). The dispatcher's instructions are in the *processor's* trace but in **no process's** trace; that is switching overhead.

State models

Two-state: Running / Not Running, with a queue feeding the dispatcher. **Limitation:** "Not Running" hides **ready** vs **blocked**, so with one queue the dispatcher may have to *search* for a runnable process, the model can't express the difference, so it can't manage it.

Five-state: New, Ready, Running, Blocked, Exit.

Transition	When
Null $\rightarrow$ New	a process is created for a program
New $\rightarrow$ Ready	OS admits it (limits exist on process count / committed virtual memory)
Ready $\rightarrow$ Running	dispatch: the dispatcher selects it
Running $\rightarrow$ Ready	timeout (max time reached) or priority preemption
Running $\rightarrow$ Blocked	waits on I/O, a system call, or IPC
Blocked $\rightarrow$ Ready	the awaited event occurs
Running $\rightarrow$ Exit	completes or aborts
Ready/Blocked $\rightarrow$ Exit	a parent terminates it (or the parent itself terminates)

Queuing model: one blocked queue is simple, but on an event the OS must **scan the whole queue** for waiters (there may be thousands). Fix: a **ready queue + one blocked queue per event**.

Swapping and suspend states

Swapping moves part or all of a process to secondary storage (a **suspend queue**) when **no resident process is Ready**. Suspension splits in two because **a suspended process may or may not still be waiting on an event**:

Ready $\rightarrow$ Ready/Suspend (suspend) · **Ready/Suspend $\rightarrow$ Ready** (activate) · **Blocked $\rightarrow$ Blocked/Suspend** (suspend) · **Blocked/Suspend $\rightarrow$ Blocked** (activate) · **Blocked/Suspend $\rightarrow$ Ready/Suspend** (event occurred while out on disk, runnable but not resident) · **Running $\rightarrow$ Ready/Suspend** · **New $\rightarrow$ Ready/Suspend**. Given a choice the OS **activates Ready/Suspend**, not Blocked/Suspend, the latter would occupy memory and still have to wait. **A suspended process (4):** not immediately available to run; may or may not be waiting on an event; was placed there by an **agent** (itself, a parent, or the OS) to prevent execution; stays until that agent explicitly releases it.

Reasons to suspend (5): swapping (free memory for a ready process); other OS reason (background/suspect process); interactive user request (debugging); timing (periodic process between intervals); parent process request.

OS control tables

Four cross-referencing groups: **memory tables** (real + virtual allocation, protection attributes, VM info), **I/O tables** (device status, buffers), **file tables** (existence, location, status), **process table** (the PCBs). They must cross-reference: the OS reaches a process's resources from its PCB and the owner from a resource.

Modes and the kernel

User mode (less privileged) vs **system/kernel mode** (more privi-

leged; privileged instructions; runs the **kernel** = the core of the OS). This protects the OS and its tables (PCBs, page tables). The PSW's **mode bit** flips to kernel on a system call or interrupt and back on return (IRT). x86 has 4 rings: **ring 0 = kernel, ring 3 = user**.

Kernel functions: *process management* (creation/termination, scheduling, dispatching, switching, synchronization + IPC, PCB management); *memory management*; *I/O management*; *support* (interrupt handling, accounting, monitoring).

Process creation and termination

Five steps: (1) assign a unique **PID**; (2) allocate space for the process; (3) initialise the **PCB**; (4) set the right **linkages** (e.g. into a queue); (5) create or expand other data structures (e.g. accounting).

Causes: new batch job; interactive log-on; created by the OS to provide a service (e.g. printing, so the user needn't wait); **spawned** by an existing process (modularity/parallelism) $\rightarrow$ **parent and child**.

Termination: halt instruction or an OS service call (generates an interrupt); log-off / terminal shutdown; quitting an application; an error or fault condition.

Interrupting a process

Mechanism	Cause	Use
Interrupt	external to the current instruction	asynchronous external event: clock, I/O, memory fault
Trap	associated with the current instruction	error/exception inside the running process; the OS decides if it is fatal (fatal $\rightarrow$ Exit + process switch)
Supervisor call	explicit request	a call to an OS function

Time slice = the max time a process may execute before being interrupted.

Mode switch vs process switch

Mode switch is cheaper: if an interrupt is pending the processor sets the PC to the handler's address and switches user $\rightarrow$ kernel mode; if the same process is still best, its context is restored and the **running process never changed**. (No interrupt pending $\implies$ just fetch the next instruction.)

Process switch: 7 steps: (1) save the processor context; (2) update the PCB of the process now leaving Running; (3) move that PCB to the appropriate queue (Ready / Blocked / Ready-Suspend); (4) select another process; (5) update *its* PCB; (6) update memory-management data structures; (7) restore the processor context to that process's saved context.

Traps & distinctions

- **Internal vs external frag.** Internal = waste *inside* an allocation (fixed partitioning, last page). External = unusable holes *between* allocations (dynamic partitioning, segmentation).
- **Logical vs relative vs physical.** Relative is a *kind* of logical address (relative to the program origin). The processor translates relative $\rightarrow$ physical, not the other way round.
- **Paging vs segmentation.** Paging: equal fixed sizes, invisible to the programmer, internal frag. Segmentation: variable sizes, visible, external frag. Segments are *not* all the same length.
- **Page vs frame.** Pages are chunks of a *process*; frames are chunks of *memory*. Pages need not land in contiguous frames.
- **Swapping vs prepaging.** Swapping moves a process out to disk; prepaging brings extra pages *in*. Not the same thing.
- **Interrupt vs trap.** External to the instruction vs caused by it.
- **Mode switch vs process switch.** A mode switch need not change the running process.
- **Blocked vs Ready.** Blocked is waiting for an *event*; Ready is waiting only for the *processor*. A timeout leaves a process Ready, not Blocked.
- **Main vs secondary memory.** Main is volatile, fast, expensive. Secondary is non-volatile, slow, cheap, *not* "fast access at relatively high cost".
- **Relocation under fixed partitioning** is required only *some-times*; under dynamic partitioning, *always*.